CSE 211 — Theory of Computation

Programming Techniques for Turing Machines · Bit-Manipulation Machines

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Atif Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — the 15-mark bit-manipulation TM since 2017-18

Lesson 14 — Storage in the State, Tracks, Subroutines — 01∗ + 10∗, w c w, multiple tracks vs multiple tapes

Lesson 15 — Bit-Manipulation TMs: XOR, AND, OR, NOT, Equality, Addition, 2's Complement — the one design pattern behind all of them

Quick Notes — TM Techniques — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lecture - 5 (Turing Machine 2).pdf (133 pages; beamer builds). Every machine was run by a simulator against its specification on all inputs up to the stated length (bit-string machines: all equal-length pairs up to 4 bits, strings up to 6). Notation: unchecked bits 0/1, checked bits x (a 0) and y (a 1); ⊔ = blank. The slides draw checking as a second track ([∗, a]); x/y are the same idea written as single symbols. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	Questions on TM techniques / bit manipulation
2023-24	Q9 TM writing 0 after the second c if w1 = w2, else 1 (tape w1cw2c) (15)
2022-23	Q10 TM inserting the NOT of w after the c (tape wc) (15)
2021-22	Q9 TM converting a signed-magnitude binary number to 2's complement (MSB 1) (15)
2020 (Jan term)	Q9 TM computing XOR of w1cw2c (result after the second c) (15)
2019-20	Q9 TM computing AND of w1cw2c (result after the second c) (15)
2017-18	Q9 TM computing bitwise OR of N1#N2# (result after the second #) (15)
2016-17	Q5(b) TM adding 1 to a binary number N on tape as #N (20)
2011-12	Q1(b) Proof idea: every multi-track TM has an equivalent single-track TM (7)
2008-09	Q3(b) TM computing bitwise XOR of two equal-length strings (0010#1011 → 1001) (13) · Q3(c) Simulate the TM of 3(b) on 1011#0010 (8)
2007-08	Q8 TM adding two equal-length binary numbers separated by ','; only the sum remains (10)
2006-07	Q11 TM computing XOR of two equal-length binary numbers separated by ',' — result replaces the second number (10)

EXAM FOCUS ★★★: every paper since 2017-18 has ONE 15-mark TM question of this kind — OR (2017-18), XOR (Jan-2020), AND (2019-20), 2's complement (2021-22), NOT/complement after c (2022-23), equality flag (2023-24). All are the same pattern: mark a bit, remember it in the state, run to the end, write the result, return. Master Lesson 15's template; the 2006-08 versions (XOR/addition, 10 marks) are in it too. 'Proof idea: multi-track ⇒ single-track' (2011-12) is Lesson 14.

Lesson 14 — Storage in the State, Tracks, Subroutines

Slides 1–83, 132–133

A TM is exactly as powerful as a conventional computer. The tricks below add only notational convenience — none extends the basic model (sl. 2).

1. Storage in the State (sl. 3–26)

The finite control can hold a finite amount of data: think of the state as a tuple [q, A, B, C] (sl. 3–6). Example: M = (Q, {0,1}, {0,1,B}, δ, [q0,B], {[q1,B]}) remembers the first symbol it sees and checks that it never occurs again — L = 01∗ + 10∗. States {q0, q1} × {0, 1, B}: control q0 'first symbol not yet read', q1 'checking'; data = the remembered symbol.

```
1. δ([q0,B], a) = ([q1,a], a, R)   a ∈ {0,1}       remember the first symbol, move on
2. δ([q1,a], ā)  = ([q1,a], ā, R)                  skip the other symbol (ā = complement)
3. δ([q1,a], B)  = ([q1,B], B, R)                  blank reached: accept [q1,B]
No rule for δ([q1,a], a): a second occurrence → halt without accepting.   (01111 accepted, 01101
rejected)
```

```
The same machine in single-letter state names (q1_0 = [q1,0], q1_1 = [q1,1], q1B = [q1,B]):
δ(q0B, 0) = (q1_0, 0, R)
δ(q0B, 1) = (q1_1, 1, R)
δ(q1_0, 1) = (q1_0, 1, R)
δ(q1_1, 0) = (q1_1, 0, R)
δ(q1_0, ⊔) = (q1B, ⊔, R)
δ(q1_1, ⊔) = (q1B, ⊔, R)
start q0B; halting/accepting ['q1B']; ⊔ blank; x/y = checked 0/1 (or X,Y,p,q as stated)
```

2. Multiple Tracks (sl. 27–83)

Think of the tape as several tracks; the tape alphabet is a tuple with one component per track, e.g. [X, Y, Z]. Tracks do not add power; a common use is data on one track and a check mark on another. Example: L_wcw = {wcw | w ∈ (0+1)⁺}. Γ = {B, ∗} × {0, 1, c, B} — the first track holds the check mark ∗, the second the symbol; states {q1…q9} × {0, 1, B}, start [q1, B], accept [q9, B]; 14 rules:

```
1. [q1,B],[B,a] → [q2,a],[∗,a],R     pick up the first unchecked symbol a, check it
2. [q2,a],[B,b] → R                  move right looking for c
3. [q2,a],[B,c] → [q3,a],R           c found
4. [q3,a],[∗,b] → R                  skip checked symbols
5. [q3,a],[B,a] → [q4,B],[∗,a],L     first unchecked symbol equals a: check it, go left
6. [q4,B],[∗,a] → L    7. [q4,B],[B,c] → [q5,B],L
8. [q5,B],[B,a] → [q6,B],L           left block not finished
9. [q6,B],[B,a] → L    10. [q6,B],[∗,a] → [q1,B],R     back to the first unchecked symbol
11. [q5,B],[∗,a] → [q7,B],R          left block finished: check the right block
12. [q7,B],[B,c] → [q8,B],R    13. [q8,B],[∗,a] → R    14. [q8,B],[B,B] → [q9,B],R  accept
No rule when the next unchecked symbol differs, or an unchecked symbol is left after c ⇒ halt, reject.
```

```
Single-symbol version (x/y = checked 0/1; q2a, q3a remember the symbol a):
δ(q1, 0) = (q20, x, R)
δ(q20, 0,1) = (q20, same, R)
δ(q20, c) = (q30, c, R)
δ(q30, x,y) = (q30, same, R)
δ(q30, 0) = (q4, x, L)
δ(q1, 1) = (q21, y, R)
δ(q21, 0,1) = (q21, same, R)
δ(q21, c) = (q31, c, R)
δ(q31, x,y) = (q31, same, R)
δ(q31, 1) = (q4, y, L)
δ(q4, x,y) = (q4, same, L)
δ(q4, c) = (q5, c, L)
δ(q5, 0,1) = (q6, same, L)
δ(q5, x,y) = (q7, same, R)
δ(q6, 0,1) = (q6, same, L)
δ(q6, x,y) = (q1, same, R)
δ(q7, c) = (q8, c, R)
δ(q8, x,y) = (q8, same, R)
δ(q8, ⊔) = (q9, ⊔, R)
start q1; halting/accepting ['q9']; ⊔ blank; x/y = checked 0/1 (or X,Y,p,q as stated)
```

Trace on 011c011 (the slide animation, sl. 57–83): the machine checks 0 on the left, finds the matching 0 right of c, returns, checks 1, 1, then verifies no unchecked symbol remains — accept.

EXAM-READY ANSWER Proof idea: every multi-track TM has an equivalent single-track TM (2011-12 Q1(b)). A k-track TM has tape alphabet Γ1 × … × Γk. Define a single-track TM whose tape symbols are the k-tuples themselves (a finite set), with Σ-symbols identified with [a, B, …, B]. Every move of the k-track machine reads and writes a whole tuple, which is one symbol of the new alphabet — so the same δ works unchanged. Hence tracks only give a convenient view of a larger alphabet and add no power. (Contrast: multiple TAPES have independent heads — Lesson 16.)

3. Subroutines (sl. 132–133)

A subroutine is a set of states with a start state and a 'return' state with no moves. A 'call' is a transition to its start state. A TM has no return-address mechanism, so if several states call the same subroutine, make a copy of its states for each caller, each copy returning to a different state.

Practice — Lesson 14

(a) Actual — 2011-12 Q1(b). (b) Constructed (slides). (c) Constructed (slides).

(a) Give the proof idea that every multi-track TM has an equivalent single-track TM. (7)

(b) Design a TM that remembers in its finite control the first symbol of its input and accepts exactly the strings $01* + 10*$.

(c) Using a second track for check marks, design a TM for $\{wcw \mid w \in (0+1)^+\}$ and give its rules.

✓ Solution

(a) EXAM-READY ANSWER in §2.

(b) §1 with the 3-rule machine and the table above (verified on all strings up to length 9).

(c) §2: 14 rules / the single-symbol table (verified on all strings up to length 7 — exactly the strings wcw are accepted).

Lesson 15 — Bit-Manipulation TMs

Slides 84–131 (binary addition) + TF problems

1. The Template (binary addition, sl. 84–131)

Tape w1 c w2 c ␣␣␣ with w1, w2 equal-length binary numbers, **LSB first** ('for convenience'); the sum is written to the right of the second c. States are tuples [q, a, b, carry]: after reading a bit of w1 the state remembers it, then the bit of w2, then the machine runs to the first blank, writes the result and the new carry goes into the state, returns to the left end, and repeats. When the first c is met in q0 all bits are done: run to the end and write the carry.

Phase	State	Action
1	q0 [carry]	skip checked bits; read the first unchecked bit a of w1, check it, remember a, go right
2	q1/s[a]	move right over bits to the first c
3	q2/t[a]	skip checked bits of w2; read the first unchecked bit b, check it, remember a, b
4	q3/u[a,b]	run right to the first blank; write f(a, b, carry); remember the new carry
5	q4–q6/v	run left to the blank before w1
6	q7/w	q0 reads c: all of w1 done — run to the end, write the final carry, halt

δ is defined by the following rules, in which a, b and c_i each may stand for either 0 or 1.

7. $\delta([q_3, a, b, c_i], [B, B]) = ([q_4, B, B, c_o], [B, s], L)$.
   - When it finds a blank, it writes the sum s to the tape and saves the carry out c_o in the state
   - s and c_o are given by

a	b	c_i	s	c_0
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure 15.1 — The slide's rule 7: write the sum bit s and remember the carry c_out (sl. 122)

$$[q_8, B, B, B]$$

✓	✓	✓		✓	✓	✓					
1	0	1	c	0	1	1	c	1	1	0	1

Figure 15.2 — Tape after the whole run on 101 c 011 c: sum 1101 (LSB first) (sl. 111)

```
Addition TM (LSB first): states p_c = scanning for the next unchecked bit of w1 with carry c; s,t,u remember a,b;
v returns
δ(p0, x,y) = (p0, same, R)
δ(p0, c) = (w0, c, R)
δ(w0, 0,1,x,y,c) = (w0, same, R)
δ(w0, ⊔) = (qh, 0, R)
δ(p0, 0) = (s00, x, R)
δ(s00, 0,1,x,y) = (s00, same, R)
δ(s00, c) = (t00, c, R)
δ(t00, x,y) = (t00, same, R)
δ(t00, 0) = (u000, x, R)
δ(u000, 0,1,x,y,c) = (u000, same, R)
δ(u000, ⊔) = (v0, 0, L)
δ(t00, 1) = (u001, y, R)
δ(u001, 0,1,x,y,c) = (u001, same, R)
δ(u001, ⊔) = (v0, 1, L)
δ(p0, 1) = (s01, y, R)
δ(s01, 0,1,x,y) = (s01, same, R)
δ(s01, c) = (t01, c, R)
δ(t01, x,y) = (t01, same, R)
δ(t01, 0) = (u010, x, R)
δ(u010, 0,1,x,y,c) = (u010, same, R)
δ(u010, ⊔) = (v0, 1, L)
δ(t01, 1) = (u011, y, R)
δ(u011, 0,1,x,y,c) = (u011, same, R)
δ(u011, ⊔) = (v1, 0, L)
δ(p1, x,y) = (p1, same, R)
δ(p1, c) = (w1, c, R)
δ(w1, 0,1,x,y,c) = (w1, same, R)
δ(w1, ⊔) = (qh, 1, R)
δ(p1, 0) = (s10, x, R)
δ(s10, 0,1,x,y) = (s10, same, R)
δ(s10, c) = (t10, c, R)
δ(t10, x,y) = (t10, same, R)
δ(t10, 0) = (u100, x, R)
δ(u100, 0,1,x,y,c) = (u100, same, R)
δ(u100, ⊔) = (v0, 1, L)
δ(t10, 1) = (u101, y, R)
δ(u101, 0,1,x,y,c) = (u101, same, R)
δ(u101, ⊔) = (v1, 0, L)
δ(p1, 1) = (s11, y, R)
δ(s11, 0,1,x,y) = (s11, same, R)
δ(s11, c) = (t11, c, R)
δ(t11, x,y) = (t11, same, R)
δ(t11, 0) = (u110, x, R)
δ(u110, 0,1,x,y,c) = (u110, same, R)
δ(u110, ⊔) = (v1, 0, L)
δ(t11, 1) = (u111, y, R)
δ(u111, 0,1,x,y,c) = (u111, same, R)
δ(u111, ⊔) = (v1, 1, L)
δ(v0, 0,1,x,y,c) = (v0, same, L)
δ(v0, ⊔) = (p0, ⊔, R)
δ(v1, 0,1,x,y,c) = (v1, same, L)
δ(v1, ⊔) = (p1, ⊔, R)
start p0; halting/accepting ['qh']; ⊔ blank; x/y = checked 0/1 (or X,Y,p,q as stated)
```

Run: 101c011c → tape yxycxyyc1101 (72 moves, halted in qh). (101 = 5 and 011 = 3 read LSB first: 1+0·2+1·4 = 5, 0+1·2+1·4 = 6 → 5 + 6 = 11 = 1011; LSB first '1101' ✓ — verified for all pairs up to 4 bits.)

2. Bitwise XOR / AND / OR (★★★ 2017-18, Jan-2020, 2019-20)

Same template without carry: f(a, b) = a ⊕ b, a ∧ b or a ∨ b, written after the second separator in the order of the bits. Tape w1 c w2 c (c or # as the separator). Only the table of f changes — the machine below is generated from f and verified for every pair of equal-length strings up to 4 bits for all three operations.

a	b	XOR	AND	OR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	0	1	1

```
TM for XOR on w1 c w2 c (result after the second c): for AND/OR replace the written bit by f(a, b)
δ(q0, x,y) = (q0, same, R)
δ(q0, 0) = (s0, x, R)
δ(q0, 1) = (s1, y, R)
δ(q0, c) = (qh, c, R)
δ(s0, 0,1,x,y) = (s0, same, R)
δ(s0, c) = (t0, c, R)
δ(t0, x,y) = (t0, same, R)
δ(t0, 0) = (u00, x, R)
δ(u00, 0,1,x,y,c) = (u00, same, R)
δ(u00, ⊔) = (v, 0, L)
δ(t0, 1) = (u01, y, R)
δ(u01, 0,1,x,y,c) = (u01, same, R)
δ(u01, ⊔) = (v, 1, L)
δ(s1, 0,1,x,y) = (s1, same, R)
δ(s1, c) = (t1, c, R)
δ(t1, x,y) = (t1, same, R)
δ(t1, 0) = (u10, x, R)
δ(u10, 0,1,x,y,c) = (u10, same, R)
δ(u10, ⊔) = (v, 1, L)
δ(t1, 1) = (u11, y, R)
δ(u11, 0,1,x,y,c) = (u11, same, R)
δ(u11, ⊔) = (v, 0, L)
δ(v, 0,1,x,y,c) = (v, same, L)
δ(v, ⊔) = (q0, ⊔, R)
start q0; halting/accepting ['qh']; ⊔ blank; x/y = checked 0/1 (or X,Y,p,q as stated)
```

Examples: 1011c0110c → tape yxyycxyyxc1101 (105 moves, halted in qh) · 1011c0110c → tape yxyycxyyxc0010 (105 moves, halted in qh) · 1011#0110# → tape yxyy#xyyx#1111 (105 moves, halted in qh)

3. XOR with the Result Replacing the Second Number (2006-07 Q11, 2008-09 Q3)

Tape w1#w2; the result overwrites w2 (p = result 0, q = result 1 as temporary marks so a processed bit of w2 is not read again), then w1 and the separator are erased and p/q turned into 0/1. For 0010#1011 the output is 1001 (the question's example; 1011#0010 gives 1001 too).

```
TM for w1#w2 → w1 ⊕ w2 (only the result left)
δ(q0, x,y) = (q0, same, R)
δ(q0, 0) = (s0, x, R)
δ(q0, 1) = (s1, y, R)
δ(q0, #) = (c1, #, R)
δ(s0, 0,1,x,y) = (s0, same, R)
δ(s0, #) = (t0, #, R)
δ(t0, p,q) = (t0, same, R)
δ(t0, 0) = (v, p, L)
δ(t0, 1) = (v, q, L)
δ(s1, 0,1,x,y) = (s1, same, R)
δ(s1, #) = (t1, #, R)
δ(t1, p,q) = (t1, same, R)
δ(t1, 0) = (v, q, L)
δ(t1, 1) = (v, p, L)
δ(v, 0,1,x,y,p,q,#) = (v, same, L)
δ(v, ⊔) = (q0, ⊔, R)
δ(c1, p,q) = (c1, same, R)
δ(c1, ⊔) = (c2, ⊔, L)
δ(c2, p) = (c2, 0, L)
δ(c2, q) = (c2, 1, L)
δ(c2, #) = (c3, ⊔, L)
δ(c3, 0,1,x,y) = (c3, ⊔, L)
δ(c3, ⊔) = (c4, ⊔, R)
δ(c4, ⊔) = (c4, ⊔, R)
δ(c4, 0,1) = (qh, same, R)
start q0; halting/accepting ['qh']; ⊔ blank; x/y = checked 0/1 (or X,Y,p,q as stated)
```

Runs: 0010#1011 → tape 1001 (86 moves, halted in qh) · 1011#0010 → tape 1001 (86 moves, halted in qh)

4. Complement: w c → w c w̄ (★★★ 2022-23 Q10)

Mark the first unchecked bit (x/y), remember its complement, run to the first blank and write it, return; when q0 meets c all bits are done: walk back left restoring x → 0, y → 1 and halt.

```
TM that appends the bitwise NOT of w after the c
δ(q0, x,y) = (q0, same, R)
δ(q0, 0) = (s0, x, R)
δ(q0, 1) = (s1, y, R)
δ(q0, c) = (qr, c, L)
δ(s0, 0,1,x,y,c) = (s0, same, R)
δ(s0, ⊔) = (v, 1, L)
δ(s1, 0,1,x,y,c) = (s1, same, R)
δ(s1, ⊔) = (v, 0, L)
δ(v, 0,1,x,y,c) = (v, same, L)
δ(v, ⊔) = (q0, ⊔, R)
δ(qr, x) = (qr, 0, L)
δ(qr, y) = (qr, 1, L)
δ(qr, ⊔) = (qh, ⊔, R)
start q0; halting/accepting ['qh']; ⊔ blank; x/y = checked 0/1 (or X,Y,p,q as stated)
```

Run: 10110c → tape 10110c01001 (102 moves, halted in qh)

5. Equality Flag: w1 c w2 c → write 0 if w1 = w2, else 1 (★★★ 2023-24 Q9)

Compare bit pairs: remember a, check it, compare with the first unchecked bit of w2. Equal → check it and return; different → run to the end and write 1; all pairs equal (q0 reaches c) → run to the end and write 0.

```
TM writing 0 after the second c if w1 = w2, else 1
δ(q0, x,y) = (q0, same, R)
δ(q0, 0) = (s0, x, R)
δ(q0, 1) = (s1, y, R)
δ(q0, c) = (w0, c, R)
δ(w0, 0,1,x,y,c) = (w0, same, R)
δ(w0, ⊔) = (qh, 0, R)
δ(s0, 0,1,x,y) = (s0, same, R)
δ(s0, c) = (t0, c, R)
δ(t0, x,y) = (t0, same, R)
δ(t0, 0) = (v, x, L)
δ(t0, 1) = (m, same, R)
δ(s1, 0,1,x,y) = (s1, same, R)
δ(s1, c) = (t1, c, R)
δ(t1, x,y) = (t1, same, R)
δ(t1, 1) = (v, y, L)
δ(t1, 0) = (m, same, R)
δ(v, 0,1,x,y,c) = (v, same, L)
δ(v, ⊔) = (q0, ⊔, R)
δ(m, 0,1,x,y,c) = (m, same, R)
δ(m, ⊔) = (qh, 1, R)
start q0; halting/accepting ['qh']; ⊔ blank; x/y = checked 0/1 (or X,Y,p,q as stated)
```

Runs: 101c101c → tape yxycyxyc0 (45 moves, halted in qh) · 101c100c → tape yxycyx0c1 (31 moves, halted in qh)

6. Signed Magnitude → 2's Complement (★★★ 2021-22 Q9)

A signed-magnitude number has sign bit s followed by the magnitude m. If s = 0 (non-negative) the 2's-complement form is identical — halt. If s = 1 the 2's-complement form keeps s = 1 and replaces the magnitude bits by their 2's complement: invert every magnitude bit and add 1 (carry out of the top is dropped). The sign bit is marked S while processing so that the increment cannot propagate into it.

```
TM for signed magnitude → 2's complement
δ(q0, 0) = (qh, 0, R)
δ(q0, 1) = (q1, S, R)
δ(q1, 0) = (q1, 1, R)
δ(q1, 1) = (q1, 0, R)
δ(q1, ⊔) = (q2, ⊔, L)
δ(q2, 1) = (q2, 0, L)
δ(q2, 0) = (q3, 1, L)
δ(q2, S) = (qh, 1, R)
δ(q3, 0,1) = (q3, same, L)
δ(q3, S) = (qh, 1, R)
start q0; halting/accepting ['qh']; ⊔ blank; x/y = checked 0/1 (or X,Y,p,q as stated)
```

Runs: 01101 → tape 01101 (1 moves, halted in qh) · 1011 → tape 1101 (9 moves, halted in qh) · 1000 → tape 1000 (9 moves, halted in qh) (−3 = 1011 in signed magnitude → 1101 in 2's complement ✓; −0 = 1000 → 1000.)

Exam Approach — The 15-Mark TM Design

1. Fix the tape format (separator, LSB or MSB first, where the answer goes); say 'assume equal length' if the question does.
2. State the idea in 3–4 lines: one round = one bit position; the bit is remembered in the state.
3. Name the states by role: scan for the next unchecked bit (q0), carry the bit right (s_a), find the partner (t_a), write at the end (u), return (v).
4. Write the rules in a table (all symbols covered by one row); draw or list the diagram.
5. Trace the small example given in the question, ID by ID or tape by tape.

TRAP Do not read a bit twice: always CHECK (mark) it. Return to the LEFT END via the blank. Make the final step explicit (halt state / write the carry). Equal-length numbers are an assumption in the question — use it.

Practice — Lesson 15

(a) Actual — 2017-18 Q9. (b) Actual — Jan-2020 Q9. (c) Actual — 2019-20 Q9. (d) Actual — 2022-23 Q10. (e) Actual — 2023-24 Q9. (f) Actual — 2021-22 Q9. (g) Actual — 2007-08 Q8. (h) Actual — 2006-07 Q11, 2008-09 Q3(b)–(c).

(a) TM taking two numbers N1 and N2 of equal length in binary and computing their logical OR; the tape initially holds N1#N2#, where # is a separator; terminate with the OR after the second # (multiple tracks and storage in the state allowed). (15)

(b) The same for XOR with tape w1 c w2 c. (15)

(c) The same for AND with tape w1 c w2 c. (15)

(d) Tape wc (w a binary string): design a TM that inserts the bitwise NOT of w after the c. (15)

(e) Tape w1 c w2 c (equal length): write 0 after the second c if w1 = w2 and 1 otherwise. (15)

(f) Design a TM that converts a signed-magnitude binary number whose MSB is 1 to 2's complement. (15)

(g) TM that adds two equal-length binary numbers separated by a comma; only the sum remains. (10)

(h) TM computing the XOR of two equal-length binary numbers separated by '#' and replacing the second number by the result; simulate 1011#0010. (10 + 8)

✓ Solution

(a)–(c) §2 with f = OR / XOR / AND — the machines above (verified for all pairs up to 4 bits). Working principle: check the leftmost unchecked bit of N1, remember it; at the second number check its leftmost unchecked bit; at the first blank write a ∨ b (a ⊕ b, a ∧ b); return; stop at the separator.

(d) §4 machine. wc = 10110c → 10110c01001.

(e) §5 machine. 101c101c → …c0; 101c100c → …c1.

(f) §6 machine.

(g) §1 (the addition machine, LSB first, sum after the second c; delete w1 c w2 c afterwards by a final sweep if only the sum may remain). If the numbers are MSB first, first reverse them or perform the addition from the right ends by marking bits from the right instead of the left — the machine is the mirror image.

(h) §3 machine. Simulation of 1011#0010: the helper marks x/y are written on w1, the results p/q on w2; after the 4 rounds the tape holds xyxy#pqqp ... then the clean-up erases w1 and #, giving 1001 (86 moves).

Quick Notes — TM Techniques

Revision points for Lessons 14–15.

L14 · Techniques

- Storage in the state: state = [q, data]; 01∗+10∗ remembers the first symbol.
- Tracks = tuples as tape symbols; no extra power (single-track proof: use the tuples as symbols).
- wcw: 14 rules, check marks ∗; subroutines = copies of state sets.

L15 · Bit manipulation

- Pattern: check bit a, remember; find partner b, check; run to blank; write f(a,b); return to left blank; repeat; halt at the separator.
- Addition LSB first: state [q, a, b, carry]; write the final carry at the end.
- NOT after c: write complement, finally restore x→0, y→1. Equality: mismatch → write 1, match to the end → write 0.
- 2's complement of signed magnitude: if s = 1 invert magnitude and add 1 (mark the sign S).

Key Diagrams to Reproduce

δ is defined by the following rules, in which a, b and c_i each may stand for either 0 or 1.

7. $\delta([q_3, a, b, c_i], [B, B]) = ([q_4, B, B, c_o], [B, s], L)$.
   - When it finds a blank, it writes the sum s to the tape and saves the carry out c_o in the state
   - s and c_o are given by

a	b	c_i	s	c_0
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Rule 7 of the addition TM (sl. 122)